RutaSmart: Asistente Inteligente para Selección de Alternativas de Transporte hacia un Destino

Sarah Isabella Peña Cortes *Filiación Completa*
(Universidad San Buenaventura, sipenacacademia.usbbog.edu.co)

***Resumen*—La creciente oferta de medios de transporte urbano (bus, metro, taxi, bicicleta, caminata y modalidades combinadas) dificulta que una persona identifique, de forma rápida y objetiva, cuál es la mejor alternativa para llegar a un destino según sus propias prioridades de tiempo, costo, comodidad o impacto ambiental. Este documento presenta el diseño de RutaSmart, una aplicación móvil desarrollada en Kotlin para Android que permite registrar rutas, paradas y medios de transporte mediante estructuras de datos y arreglos, comparar alternativas por medio de ciclos de control, y representar los elementos del dominio (rutas, estaciones, medios de transporte, usuarios) mediante programación orientada a objetos. La solución incorpora un asistente basado en inteligencia artificial que analiza las alternativas disponibles y recomienda la ruta más adecuada de acuerdo con los criterios definidos por el usuario. Se documentan además el manejo de excepciones, la seguridad contra valores nulos propia de Kotlin, el proceso de depuración planificado, la proyección de una implementación multiplataforma mediante Kotlin Multiplatform Mobile y el plan de pruebas de funcionamiento. Como resultado se espera contar con un producto inicial —el presente documento— que sustente técnica y metodológicamente el desarrollo posterior de la aplicación funcional.**

V. Introducción

La movilidad en ciudades intermedias y grandes se ha vuelto cada vez más compleja: una misma persona puede llegar a un destino caminando, en bicicleta, en bus urbano, en sistemas de transporte masivo tipo metro o BRT, en taxi o mediante plataformas de transporte por aplicación, o incluso combinando varios de estos medios en un mismo trayecto. Cada alternativa implica diferencias en tiempo de viaje, costo económico, número de transbordos, nivel de comodidad y huella ambiental, y estas diferencias no siempre son evidentes para la persona que debe decidir en el momento.

Ante este panorama, resulta pertinente el desarrollo de una aplicación móvil que centralice la información de rutas, paradas y medios de transporte disponibles, y que —más allá de simplemente mostrar opciones— sea capaz de compararlas de forma sistemática y de recomendar la alternativa más adecuada según las preferencias particulares de cada usuario. Este proyecto propone RutaSmart, una solución para Android desarrollada en Kotlin que integra conceptos fundamentales de programación (estructuras de datos, arreglos, ciclos y programación orientada a objetos) con un componente de inteligencia artificial que actúa como asistente de decisión.

El presente documento constituye el producto inicial del proyecto: un diseño técnico y metodológico completo, en formato IEEE, que sustenta el desarrollo posterior de la aplicación funcional.

IV. Palabras clave

Las palabras clave seleccionadas para este proyecto se listan igualmente en la sección de encabezado del documento e incluyen: Kotlin, aplicaciones móviles, Android Studio, inteligencia artificial, programación orientada a objetos, estructuras de datos, arreglos, ciclos, movilidad urbana y Kotlin Multiplatform.

VI. Pregunta problema

¿Cómo puede una aplicación móvil desarrollada en Kotlin ayudar a una persona a seleccionar, de forma objetiva y personalizada, la alternativa de transporte más adecuada para llegar a un destino, considerando las rutas, paradas y medios de transporte disponibles y los criterios propios del usuario?

VII. Descripción de la necesidad

Las personas que se movilizan diariamente en entornos urbanos —estudiantes, trabajadores, adultos mayores, visitantes ocasionales de una ciudad— enfrentan con frecuencia la dificultad de no conocer con precisión cuál es la mejor forma de llegar a un destino cuando existen varias alternativas de transporte disponibles. Esta dificultad se agrava cuando la persona no conoce bien la ciudad, cuando las condiciones cambian (tráfico, clima, horarios de servicio) o cuando sus prioridades varían según el momento: en ocasiones prima el tiempo, en otras el costo, y en otras la comodidad o la seguridad percibida.

Los usuarios afectados por esta necesidad son, principalmente, habitantes y visitantes de zonas urbanas que dependen de transporte público o mixto para desplazarse y que no cuentan con una herramienta que compare de manera integral —y no solo geográfica— las alternativas disponibles. La necesidad que origina el proyecto es, por tanto, contar con una aplicación que no solo indique posibles rutas, sino que las

* Revista Argentina de Trabajos Estudiantiles. Patrocinada por la IEEE.

compare mediante criterios definidos por el propio usuario y le entregue una recomendación clara y justificada.

VIII. Estado del arte

Existen en el mercado aplicaciones ampliamente utilizadas para apoyar la movilidad urbana. Google Maps ofrece cálculo de rutas multimodales con estimaciones de tiempo y, en algunas ciudades, información de tarifas, pero su motor de recomendación no es configurable por el usuario según criterios explícitos de prioridad. Moovit se especializa en transporte público y muestra horarios y ocupación estimada de vehículos, aunque su enfoque de comparación entre medios de transporte alternos (por ejemplo, transporte público frente a transporte privado) es limitado. Waze se centra en la navegación vehicular en tiempo real y no contempla otros medios de transporte. Citymapper integra múltiples medios en algunas ciudades y presenta comparativas de tiempo y costo, pero su cobertura geográfica es reducida y no todos sus mercados incluyen un asistente conversacional.

A partir de esta revisión se identifica una oportunidad de diferenciación para RutaSmart: la posibilidad de que el propio usuario defina y pondere los criterios de decisión (tiempo, costo, transbordos, comodidad, impacto ambiental) y de que un asistente de inteligencia artificial explique en lenguaje natural por qué una alternativa es recomendada frente a las demás, en lugar de limitarse a mostrar una lista de opciones.

IX. Tecnologías utilizadas

A. Lenguaje y entorno de desarrollo

Kotlin será el lenguaje principal de desarrollo, por su seguridad frente a valores nulos, su soporte nativo en Android y su compatibilidad con Kotlin Multiplatform. Android Studio se utilizará como entorno integrado de desarrollo (IDE), incluyendo su depurador, el emulador de dispositivos y las herramientas de perfilado (Profiler).

B. Interfaz de usuario

Jetpack Compose se empleará para construir la interfaz declarativa de la aplicación, junto con componentes de Material Design 3 para mantener consistencia visual.

C. Datos y persistencia

Room (sobre SQLite) se utilizará para la persistencia local de rutas, paradas y preferencias del usuario. Retrofit y Kotlinx Serialization permitirán el consumo de servicios web externos de mapas y transporte.

D. Concurrencia

Kotlin Coroutines y Flow gestionarán las operaciones asíncronas, como la consulta de rutas o la comunicación con el asistente de IA, sin bloquear el hilo principal de la interfaz.

E. Mapas y geolocalización

El SDK de Google Maps for Android y los servicios de geolocalización (Fused Location Provider) permitirán ubicar al usuario y representar rutas y paradas sobre el mapa.

F. Inteligencia artificial

Se proyecta el uso de un modelo de lenguaje mediante una API de IA conversacional (por ejemplo, a través de la API de Anthropic o de un modelo equivalente) para el análisis de alternativas y la generación de recomendaciones explicadas en lenguaje natural, complementado con un motor de puntuación (scoring) propio, programado en Kotlin, para la comparación cuantitativa de rutas.

X. Diseño de la aplicación

La aplicación se proyecta con las siguientes pantallas principales:

- Pantalla de inicio: permite ingresar el origen y el destino, ya sea escribiendo la dirección o seleccionando un punto en el mapa.
- Pantalla de preferencias: el usuario define y prioriza los criterios de decisión (tiempo, costo, comodidad, número de transbordos, impacto ambiental).
- Pantalla de alternativas: presenta, en forma de lista comparativa, las diferentes opciones de transporte encontradas para el trayecto solicitado.
- Pantalla de detalle de ruta: muestra el mapa con la ruta seleccionada, las paradas intermedias y los medios de transporte involucrados en cada tramo.
- Pantalla del asistente de IA: interfaz de conversación donde el usuario puede preguntar por qué se recomienda una ruta o solicitar ajustes a la recomendación.
- Pantalla de historial: guarda las rutas consultadas anteriormente para facilitar consultas futuras.

La navegación entre pantallas seguirá un patrón de navegación por pila (Navigation Component de Jetpack), partiendo de la pantalla de inicio hacia las pantallas de preferencias, alternativas y detalle, con la posibilidad de invocar el asistente de IA desde cualquiera de ellas mediante un botón flotante.

XI. Arquitectura de la solución

La aplicación se organizará siguiendo el patrón arquitectónico MVVM (Model-View-ViewModel), recomendado por Google para aplicaciones Android, complementado con una separación por capas inspirada en la arquitectura limpia (Clean Architecture):

- Capa de presentación (UI): pantallas construidas en Jetpack Compose y ViewModels que exponen el estado mediante StateFlow.
- Capa de dominio: casos de uso (por ejemplo, ObtenerAlternativasUseCase, RecomendarRutaUseCase) y las clases del modelo (Ruta, Parada, MedioDeTransporte, Alternativa).

- Capa de datos: repositorios que abstraen el origen de la información (base de datos local Room, servicios REST de mapas, servicio de IA), aplicando el patrón Repository.

Los componentes se comunican de forma unidireccional: la vista notifica eventos al ViewModel, el ViewModel invoca los casos de uso del dominio, y estos, a través de los repositorios, obtienen o calculan la información que finalmente se refleja de nuevo en la vista mediante flujos observables.

XII. Aplicación de estructuras, arreglos y ciclos

Las estructuras de datos y los arreglos se utilizarán para representar el dominio de rutas y paradas: cada Ruta contendrá una lista (List<Parada>) con las paradas ordenadas del trayecto; un arreglo (Array<MedioDeTransporte>) almacenará los medios de transporte disponibles para un tramo; y un mapa (HashMap<String, Parada>) permitirá representar la red de paradas como un grafo, donde cada clave es el identificador de una parada y el valor contiene sus conexiones. Las alternativas de transporte generadas para un mismo trayecto se almacenarán en una MutableList<Alternativa> que el motor de recomendación recorrerá para compararlas.

Los ciclos de control se emplearán principalmente en dos procesos: (1) la construcción de alternativas, donde un ciclo for recorre las paradas de la red para identificar combinaciones válidas entre el origen y el destino; y (2) la comparación de alternativas, donde un ciclo for o forEach recorre la lista de alternativas calculando, para cada una, un puntaje ponderado según los criterios definidos por el usuario (tiempo, costo, comodidad), seleccionando finalmente la de mayor puntaje mediante una variable acumuladora. Un ciclo while se utilizará en el proceso de sincronización periódica de datos de tráfico o de horarios de transporte público, repitiéndose mientras exista conexión de red disponible.

XIII. Aplicación de programación orientada a objetos

El modelo de dominio se representará mediante clases y objetos. La clase Parada encapsula atributos como identificador, nombre, latitud y longitud. La clase Ruta agrupa una lista de paradas y expone métodos como calcularDistanciaTotal() y calcularTiempoEstimado(). La interfaz MedioDeTransporte define un contrato común (métodos calcularCosto() y calcularDuracion()) que es implementado por las clases Bus, Metro, Taxi, Bicicleta y Caminata, aplicando así el principio de polimorfismo: el motor de recomendación puede tratar cualquier medio de transporte de forma uniforme a través de la interfaz, sin conocer su tipo concreto.

La clase Alternativa agrega una Ruta con uno o varios MedioDeTransporte y calcula un puntaje mediante composición con la clase CriteriosUsuario, que encapsula las prioridades definidas por la persona. La clase AsistenteIA representa el componente responsable de invocar el servicio de inteligencia artificial y de traducir sus resultados en una recomendación legible para el usuario. Todas las clases aplican encapsulamiento (atributos privados con propiedades val/var expuestas de forma controlada) y las relaciones entre ellas siguen asociación, agregación y composición según corresponda, evitando el uso de herencia salvo en los casos donde realmente existe una relación “es un” (por ejemplo, entre las distintas implementaciones de MedioDeTransporte).

XIV. Manejo de excepciones

Se identifican los siguientes escenarios de error y su tratamiento mediante excepciones: (1) RutaNoEncontradaException, lanzada cuando no existe ninguna combinación de paradas y medios de transporte entre el origen y el destino solicitados; (2) UbicacionInvalidaException, lanzada cuando la dirección u origen/destino ingresados no pueden geocodificarse; (3) ServicioNoDisponibleException, lanzada ante fallas de red al consultar el servicio de mapas o el asistente de IA; y (4) CriteriosIncompletosException, lanzada cuando el usuario no ha definido ningún criterio de comparación.

Cada uno de estos casos se controlará mediante bloques try-catch-finally alrededor de las operaciones de red y de cálculo de alternativas, mostrando al usuario un mensaje comprensible (por ejemplo, sugiriendo verificar la conexión a internet) en lugar de exponer el error técnico, y registrando el detalle del error en un log interno para su posterior depuración.

XV. Seguridad contra valores nulos en Kotlin

Se aprovechará el sistema de tipos de Kotlin para distinguir explícitamente entre tipos nulables (String?, Parada?) y no nulables. Los atributos que provienen de fuentes externas (respuesta de servicios web, resultados de geocodificación) se declararán como nulables y se accederán mediante el operador de llamada segura (?.) y el operador Elvis (?:) para asignar valores por defecto cuando corresponda, evitando así excepciones de tipo NullPointerException. La función let se utilizará para ejecutar bloques de código únicamente cuando un valor nulable esté efectivamente presente, y requireNotNull() o checkNotNull() se emplearán en los puntos de entrada críticos (por ejemplo, antes de iniciar el cálculo de una ruta) para validar de forma temprana y explícita que los datos necesarios existen. El uso de lateinit se restringirá a propiedades que se inicializan garantizadamente antes de su primer uso, como referencias a vistas o al ViewModel.

XVI. Proceso de depuración

El proceso de depuración se apoyará en las herramientas propias de Android Studio: el depurador integrado con puntos de interrupción (breakpoints) para inspeccionar el estado de las variables durante el cálculo de alternativas; Logcat para revisar los mensajes de registro (log) generados en cada capa de la aplicación; y el Layout Inspector para verificar el comportamiento de la interfaz en Jetpack Compose. A nivel de código se emplearán pruebas unitarias con JUnit para detectar errores de forma temprana en la lógica de comparación de rutas, y se planea integrar Firebase Crashlytics para el

monitoreo de fallos una vez la aplicación se encuentre en fase de pruebas con usuarios reales.

XVII. Implementación multiplataforma

Se proyecta el uso de Kotlin Multiplatform Mobile (KMM) para compartir entre Android e iOS la lógica de negocio que no depende de la plataforma: las clases de dominio (Ruta, Parada, MedioDeTransporte, Alternativa), el motor de comparación y puntuación de alternativas, y el cliente de comunicación con el servicio de inteligencia artificial. De esta manera, únicamente las capas de interfaz de usuario y de acceso a APIs nativas (mapas, geolocalización) se implementarían de forma específica para cada plataforma, mientras que el núcleo de reglas de negocio se mantiene único y reutilizable. Adicionalmente, se contempla que este mismo módulo compartido pueda compilarse en el futuro hacia Kotlin/JS, lo que permitiría reutilizar la lógica de recomendación en una eventual versión web de RutaSmart.

XVIII. Integración o asistencia mediante inteligencia artificial

La inteligencia artificial actuará como un asistente de decisión dentro de la aplicación, y no como un simple buscador de rutas. Su función es doble: por un lado, un motor de puntuación programado en Kotlin calcula de forma determinística un puntaje numérico para cada alternativa a partir de los criterios ponderados por el usuario (tiempo, costo, comodidad, transbordos, impacto ambiental); por otro lado, un servicio de IA conversacional recibe dicho puntaje junto con los datos de cada alternativa y genera una explicación en lenguaje natural de por qué una opción es la más recomendable, permitiendo además que el usuario haga preguntas de seguimiento (“¿por qué es mejor que la ruta en bus?”, “¿cuál es más económica si viajo en horas pico?”). Este diseño combina la trazabilidad y control de un algoritmo propio con la flexibilidad conversacional de un modelo de lenguaje, evitando que la recomendación final dependa exclusivamente de una “caja negra”.

XIX. Pruebas de funcionamiento

Se plantea un plan de pruebas en tres niveles. Las pruebas unitarias (JUnit) verificarán, de forma aislada, el cálculo de distancia y tiempo de una ruta, el algoritmo de puntuación de alternativas y el correcto lanzamiento de las excepciones definidas. Las pruebas de integración verificarán la comunicación entre los repositorios y las fuentes de datos (base de datos Room y servicios REST simulados). Las pruebas de interfaz (Espresso/Compose Testing) verificarán los flujos principales de usuario, como el ingreso de origen y destino y la visualización de la pantalla de alternativas. La Tabla I resume algunos casos de prueba proyectados.

Tabla I
Resumen de casos de prueba proyectados

ID	Caso de prueba	Resultado esperado
CP-01	Buscar ruta con origen y destino válidos	Se muestra una lista de alternativas ordenada por puntaje
CP-02	Buscar ruta sin conexión a internet	Se captura ServicioNoDisponibleException y se informa al usuario
CP-03	Ingresar destino inexistente	Se lanza UbicacionInvalidaException y se solicita corregir el dato
CP-04	Definir criterios de comparación y solicitar recomendación	El asistente de IA justifica la alternativa recomendada
CP-05	Consultar historial de rutas	Se listan correctamente las rutas previamente calculadas

XX. Resultados

Dado que este documento corresponde al producto inicial del proyecto (fase de diseño), los resultados que se presentan son proyectados y se centran en los entregables de diseño solicitados: la definición completa de la necesidad y la pregunta problema, el diseño de pantallas y navegación (mockup), la arquitectura MVVM por capas, el modelo de clases orientado a objetos, la identificación concreta de estructuras de datos, arreglos y ciclos dentro del dominio del transporte, la propuesta de manejo de excepciones y de seguridad contra nulos, el plan de depuración, la propuesta de implementación multiplataforma mediante KMM, el diseño de la asistencia por IA y el plan de pruebas. Se espera que, en la fase de implementación posterior, estos diseños permitan construir una aplicación funcional en la que la comparación y recomendación de alternativas de transporte pueda validarse con datos y usuarios reales.

XXI. Conclusiones

El diseño presentado demuestra que la necesidad de apoyar a una persona en la selección de alternativas de transporte puede abordarse de manera coherente integrando conceptos fundamentales de programación: estructuras de datos y arreglos para modelar rutas y paradas, ciclos para comparar alternativas, programación orientada a objetos para representar el dominio de forma extensible, y un componente de inteligencia artificial que aporta valor real al usuario al explicar sus recomendaciones y no solo al calcularlas.

Como aprendizaje central del proceso de diseño se destaca la importancia de separar claramente la lógica de negocio (independiente de la plataforma) de la interfaz de usuario, lo cual no solo favorece la calidad del código mediante manejo adecuado de excepciones y de valores nulos, sino que además habilita, casi de forma natural, la proyección de una solución multiplataforma. El proyecto queda así en condiciones de avanzar hacia la fase de implementación en Android Studio, siguiendo la arquitectura y el modelo de clases aquí definidos.

XXII. Referencias bibliográficas

[1] JetBrains, “Kotlin Documentation,” Kotlin, 2024. [En línea]. Disponible: https://kotlinlang.org/docs/home.html

[2] Google, “Guía de Android para desarrolladores,” Android Developers, 2024. [En línea]. Disponible: https://developer.android.com/guide

[3] Google, “Jetpack Compose,” Android Developers, 2024. [En línea]. Disponible: https://developer.android.com/jetpack/compose

[4] JetBrains, “Kotlin Multiplatform,” Kotlin, 2024. [En línea]. Disponible: https://kotlinlang.org/docs/multiplatform.html

[5] Google, “Guide to app architecture,” Android Developers, 2024. [En línea]. Disponible: https://developer.android.com/topic/architecture

[6] Google, “Room persistence library,” Android Developers, 2024. [En línea]. Disponible: https://developer.android.com/training/data-storage/room

[7] Google, “Maps SDK for Android,” Google for Developers, 2024. [En línea]. Disponible: https://developers.google.com/maps/documentation/android-sdk

[8] Moovit Inc., “Moovit App,” 2024. [En línea]. Disponible: https://moovitapp.com

[9] Citymapper Ltd., “Citymapper,” 2024. [En línea]. Disponible: https://citymapper.com

[10] Google, “Firebase Crashlytics,” Firebase, 2024. [En línea]. Disponible: https://firebase.google.com/docs/crashlytics

[11] IEEE, “IEEE Editorial Style Manual,” IEEE, 2023. [En línea]. Disponible: https://www.ieee.org/authortools